Endterm Project — Resilient University Information System

Fault-Tolerant University Registration/Records Service: Gateway, Observability, Security and Fault Injection

Astana IT University | Fault Tolerance and Dependable Computing | Academic Year 2026–2027

Student: Shamil Yakupov, ID 255193 | Instructor: Azamat Serek

Project period: 28 October – 4 November 2026 | Report date: 4 November 2026

Context note: all data in this project are synthetic. The system under test is a local Kubernetes lab; no real users, production systems or real university infrastructure were involved. The application is the hoWallet Go API, used (as in the midterm) only as a stand-in for a synthetic university service.

Abstract. The endterm extends the midterm Registration Service lab into a monitored, hardened deployment: an nginx API gateway with bounded retries and JSON degradation, two API replicas with probes and drain-on-shutdown, a Prometheus metrics endpoint added to the service, three alert rules, a Grafana dashboard, request correlation across the gateway and the API, and network segmentation. Eight experiments (six with fault injection or abnormal conditions, two security/misuse tests) were run on 4 November 2026 against both the endterm configuration and a retained single-replica baseline. A replica loss cost the baseline 137–138 of 139 requests (23 s outage) and the endterm none; a 20 s database outage was answered with a JSON 503 within the 5 s gateway cap instead of 12 s hangs; every acknowledged write was stored exactly once in all integrity checks. One alert rule was found to be unable to fire, was fixed and re-verified; other limitations are reported in Section 9.

1. Introduction and Relationship to the Midterm

The midterm lab (Assignment 3) deployed a synthetic AITU **Registration Service** on a local kind cluster and measured replica loss, total service loss and database unavailability. The service was the hoWallet Go API (a personal-finance tracker written by the author) running unchanged as a stand-in application. The endterm continues exactly that system: the hoWallet API keeps the role of the university **Registration/Records** service, PostgreSQL remains its data store, and all accounts, households and transactions are synthetic lab records. The domain words in logs and tables come from the code: a *household* is the tenant (one student's record space), an *account* is a ledger, and a *transaction* is a record write (for example a fee payment or a registration entry).

What changed since the midterm (all changes are in the lab manifests, except one opt-in metrics patch to the service):

Table 1. Midterm (A3) versus endterm configuration.

Aspect	Midterm (A3 lab)	Endterm (this report)
Entry point	Kubernetes Service / NodePort straight to the API	nginx gateway x2 (NodePort 30080): timeouts, bounded retries, JSON 503/413, correlation id, client-IP sanitising
API replicas	2, readiness/liveness on /health, preStop added at the end of A3	2, maxUnavailable 0, preStop sleep 5, anti-affinity, hardened pod (non-root, read-only rootfs)
Observability	kubectl and client-side probe only	GET /metrics in the API (feat/metrics), Prometheus (2 s scrape), Grafana dashboard, 3 alert rules, JSON logs joined by request id
Security	application controls only	+ NetworkPolicies (default-deny ingress, API egress allow-list), gateway body limit, X-Real-IP overwrite, threat model
Testing	3 manual failure simulations	8 scripted experiments with acceptance criteria, 2–6 repetitions, integrity checks in SQL
Baseline kept	–	api-baseline: 1 replica, no preStop, no gateway, direct NodePort 30081, same image and database

The baseline deliberately keeps the midterm's direct exposure and drops the A3 hardening (one replica, no preStop), so that it represents the system before the resilience work and the comparison isolates the endterm mechanisms. Both configurations run side by side in one cluster, use the same image (`howallet-api:endterm`) and the same database, and are probed at the same time where possible.

1.1 Rubric and requirement traceability

Table 2. Traceability: requirement → report section → evidence (paths relative to the repository root).

Requirement (assignment section)	Section	Evidence file(s)
Containerised services, replicas, health checks, restart strategy (5)	3.1, 3.5	k8s/30-api.yaml, evidence/02-get-all.txt
API gateway, load balancing, timeouts, bounded retries (3, 5)	3.3	k8s/40-gateway.yaml
Service discovery / configuration (3)	3.1	k8s/30-api.yaml (headless Service), k8s/50-prometheus.yaml
Graceful degradation of a non-critical feature (5)	3.4, 8.7	evidence/e8-graceful-degradation.txt
Instance failure demonstrated (3)	8.2	evidence/e1-replica-failure.txt, results/e1-*.json
≥ 3 metrics (5)	4.1	patch/feat-metrics.diff, evidence/03-targets.txt
Structured logs, correlation ids (5)	4.2	evidence/04-correlation.txt, e5-access-control.txt
Dashboard (3)	4.3	k8s/60-grafana.yaml, screenshots/grafana-*.png
≥ 3 alerts with threshold and response (5)	4.4	k8s/50-prometheus.yaml, evidence/05-alert-rules.txt
Threat model (3)	2.3	notes/threat-model.md
≥ 3 security controls (5)	5	notes/security-controls.md, k8s/70-network-policy.yaml
≥ 2 security / misuse tests (6)	8.6	evidence/e5-*.txt, e6-*.txt
Automated tests (5)	6.1	notes/test-inventory.md, patch/feat-metrics.diff
Repeatable fault-injection scripts (5)	7, App. A	scripts/faults.py, scripts/exps.py, scripts/ftlab.py
≥ 6 experiments, ≥ 4 fault, ≥ 2 security (6)	7.4	evidence/results/*.json
Required measurements (7)	8.1–8.9	evidence/results/summary.md, summary-data.md
Alerts triggered and correctness (7)	8.5	evidence/e4-alerts.txt, results/e4-*.json
Data integrity (7)	8.3, 8.7, 8.8	results/e2-*.json, e7-*.json, evidence/99-final-integrity.txt
Baseline vs endterm (7)	8	k8s/baseline/api-baseline.yaml, all e1/e2/e3/e7 results
Demonstration of ≥ 3 scenarios (9)	App. A	scripted: E1 (failure), E5/E6 (security), E4 (alert)

2. Updated Requirements, Assumptions and Threat Model

2.1 Requirements and acceptance criteria

Table 3. Dependability requirements and their measurable acceptance criteria.

ID	Requirement	Acceptance criterion (checked in)
R1	Loss of one API instance is invisible to clients	0 failed probe requests, replica replaced < 10 s (E1)
R2	A database outage fails fast and honestly	JSON error, latency bounded (≤ 5 s gateway cap), alert fires, service recovers (E2)
R3	Capacity for a documented load step	≥ 99 % success at 30 req/s, p95 < 500 ms, no alert (E3)
R4	Incidents are detected by alerts, not by users	each rule fires on its scenario, none fires in a quiet window (E4)
R5	No access to another tenant's records	401 without token, 404 for a foreign household, data unchanged, audit trail per request (E5)
R6	Malformed or abusive input does not destabilise the service	400/413 for bad input, lockout 429 with Retry-After, probe unaffected (E6)
R7	No lost or duplicate records when a write is interrupted	rows = 200, 0 duplicates, balance = −sum after a pod kill (E7)
R8	A failing non-critical dependency degrades one feature only	same-currency writes 201, foreign-currency 422 `no_rate`, nothing wrong written (E8)

2.2 Assumptions

- Single-node kind cluster (Kubernetes v1.33.1) on Docker Desktop/WSL2; node-level and zone failures are out of scope. One PostgreSQL instance (no replica) — database failover is not claimed.
- Clients reach the cluster through NodePorts on 127.0.0.1; the kind node SNATs them, so the API sees one client address (`10.244.0.1`). Per-IP limits are therefore shared by every test client.
- The exchange-rate provider (Frankfurter) is a non-critical external dependency; transactions in the base currency must not depend on it.
- Secrets in the repository are synthetic lab placeholders (Kubernetes Secret `university-secrets`).

2.3 Threat model (summary)

The full STRIDE analysis with code references is in `notes/threat-model.md`. Assets: financial/academic records and balances, credentials (bcrypt cost 12, refresh tokens stored as SHA-256), the JWT signing secret, tenant membership, the audit trail. Actors: A1 anonymous attacker, A2 authenticated user of another tenant, A3 compromised pod, A4 abusive or buggy client.

Table 4. Threats per entry point, implemented controls and residual risk (condensed).

Entry point / actor	Threat (STRIDE)	Control	Residual risk
/auth/* (A1)	S: brute force, credential stuffing	10 req/min per IP on /auth; lockout after 5 failures per (e-mail, IP) for 15 min; bcrypt	counters live in each replica's memory (E6: 10 attempts with 2 replicas)
/auth/* (A1)	S/T: forged JWT	HMAC only, issuer and exp required; same 401 for all failures	whoever holds JWT_SECRET can mint tokens (A3)
Household API (A2)	I/T: IDOR via X-Household-ID or foreign ids	membership check → 404; queries scoped by household_id	– (tested in E5)
Household API (A4)	T: duplicate writes on retry	client_id idempotency key + unique index	only POST /api/transactions has it
Household API (A4)	D: flooding, huge bodies	600/min per user, 1200/min per IP, 60/min creates; 2 MB gateway, 1 MB decode	per-replica limits; no statement_timeout
Gateway (A1)	S: spoofed X-Real-IP to dodge limits	gateway overwrites X-Real-IP, clears True-Client-IP	baseline (no gateway) trusts the header (E6 finding)
Any pod (A3)	E/I: lateral movement	NetworkPolicies, non-root, read-only rootfs, caps dropped	API DB user owns all tables; logs in the same DB
All (A1–A4)	R: deny actions	JSON request log with request_id/user_id; audit_logs; app_logs	no dedicated failed-login event; audit queue drops under load

3. Cloud/Microservices Architecture and Resilience Design

3.1 Components

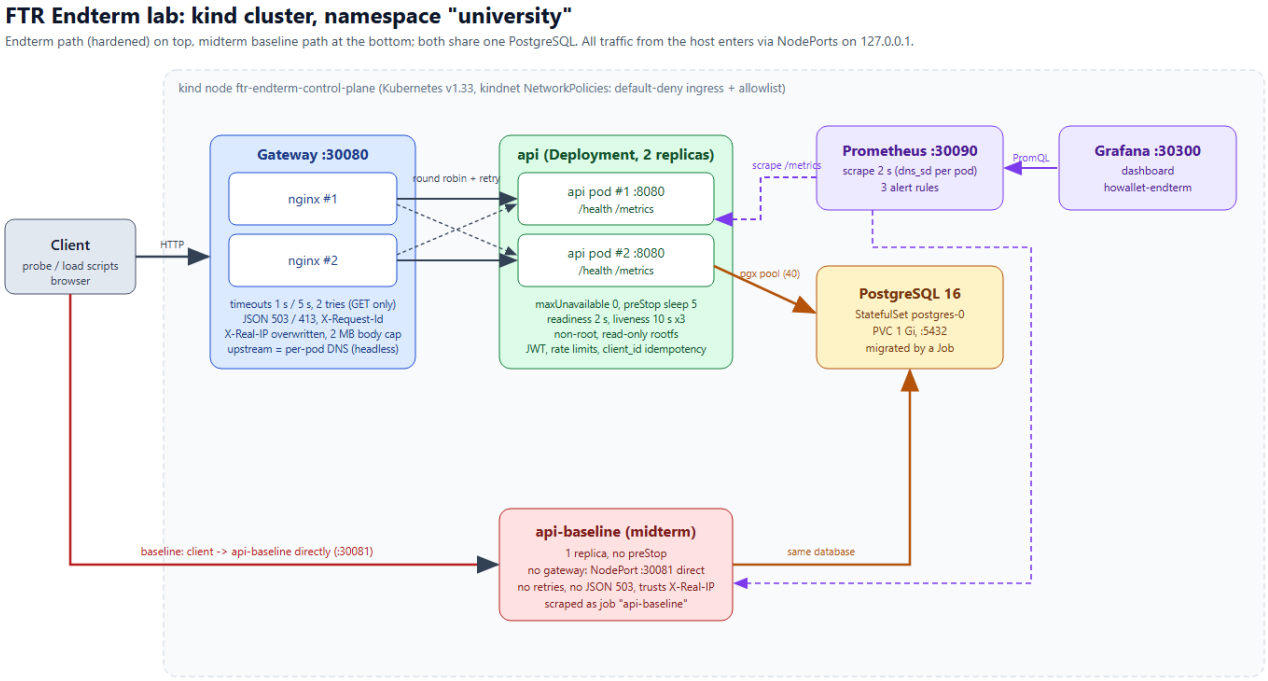


Figure 1. Endterm lab architecture: hardened path (gateway → api x2 → PostgreSQL) with monitoring, and the retained midterm baseline path (bottom) sharing the same database.

The system has six independently deployed components in namespace `university` (Table 5). Service discovery is DNS-based: the gateway and Prometheus resolve the headless Service `api-headless`, which returns one A record per ready API pod; configuration comes from ConfigMaps (gateway, Prometheus, Grafana) and one Secret.

Table 5. Components, replication and recovery strategy.

Component	Kind / replicas	Health checking	Recovery strategy
gateway (nginx 1.27)	Deployment x2, NodePort 30080	readiness/liveness GET /healthz	restart by kubelet; the other gateway pod keeps serving
api (hoWallet Go API)	Deployment x2, RollingUpdate maxUnavailable 0 / maxSurge 1	readiness /health every 2 s (1 failure); liveness every 10 s (3 failures)	ReplicaSet replaces a deleted pod; liveness restarts a hung one; preStop sleep 5 drains
postgres 16	StatefulSet x1, PVC 1 Gi	readiness pg_isready	restart in place; data on PVC
university-migrate	Job	–	runs migrations once before the API
prometheus 2.54.1	Deployment x1	/-/ready	stateless (1 h retention), restart
grafana 11.2.0	Deployment x1	–	dashboard provisioned from ConfigMap
api-baseline	Deployment x1, NodePort 30081	readiness /health only	replacement pod only

3.2 Resilience mechanisms

Table 6. Resilience mechanisms, their configuration and where they were exercised.

Mechanism	Configuration	Purpose	Exercised in
Redundancy	2 API and 2 gateway replicas, anti-affinity (preferred)	survive an instance loss	E1, E4b, E7
Drain on shutdown	preStop `sleep 5`, terminationGracePeriodSeconds 20, server Shutdown 10 s	finish in-flight requests while endpoints update	E1 graceful
Fast removal	readiness period 2 s, failureThreshold 1	stop routing to a sick pod	E1, E4b
Timeouts	gateway connect 1 s, read 5 s; API server read/write 15	bound latency, Slowloris	E2, E4b

Mechanism	Configuration	Purpose	Exercised in
	s, header 5 s		
Bounded retries	`proxy_next_upstream error timeout http_502/503/504`, tries 2, idempotent methods only	hide one bad replica from GETs	E1, E4b
Passive circuit breaking	`max_fails=1 fail_timeout=2s` per upstream server	skip a failed replica for 2 s	E1, E2
Graceful degradation	JSON 503 + Retry-After 2 (gateway); rates archive with 422 `no_rate`	honest, machine-readable failure	E2, E8
Idempotent writes	client_id + unique index (household_id, client_id)	safe client retries	E2, E7
Segmentation	NetworkPolicies: default-deny ingress, API egress allow-list	contain a compromised pod	deployment (A4 method)

3.3 Timeout, retry and circuit-breaker policy

The policy follows the stability patterns of Nygard [10]: every remote call has a timeout, retries are bounded and only for idempotent requests, and a failing instance is taken out of rotation. nginx resolves `api-headless` every 2 s (`server … resolve`, nginx ≥ 1.27.3) so that each API pod is a separate upstream server; with the single ClusterIP there would be no "next upstream" to retry on [8, 9]. POST is never replayed by the gateway (`non_idempotent` is deliberately absent); clients retry writes themselves with the same `client_id`. Inside the API there is no generic retry library and no circuit breaker; the database pool (pgx, 40 connections per replica) has no acquire timeout and PostgreSQL has no `statement_timeout`, so a blocked database shows up as requests waiting until the gateway's 5 s read timeout (Section 9).

```
upstream api_upstream {
  zone api_upstream 64k;
  server api-headless.university.svc.cluster.local:8080 resolve max_fails=1 fail_timeout=2s;
}
location / {
  proxy_set_header X-Real-IP $remote_addr;   proxy_set_header True-Client-IP "";
  proxy_set_header X-Request-Id $req_id;     # adopted or minted correlation id
  proxy_connect_timeout 1s;  proxy_read_timeout 5s;
  proxy_next_upstream error timeout http_502 http_503 http_504;  proxy_next_upstream_tries 2;
  error_page 502 503 504 = @degraded;   error_page 413 = @too_large;   # JSON bodies
}
```

Listing 1. Gateway excerpt (k8s/40-gateway.yaml).

3.4 Graceful degradation

Two degradations are designed in. (1) **Gateway:** when no upstream answers in time, the client gets `503 {"error":"service_unavailable",…,"request_id":…}` with `Retry-After: 2` instead of an HTML page or a hung connection. (2) **Exchange rates (non-critical feature):** the write path never calls the external provider; a background job fills an archive table. When the provider is unreachable, writes in the base currency are unaffected, while a foreign-currency write with no archived rate is refused with `422 no_rate` and nothing is written — the service never invents a rate of 1 or a zero (E8).

3.5 Deployment and scaling strategy

Rolling updates keep two ready API pods (`maxUnavailable: 0`, `maxSurge: 1`). Resource requests/limits: API 50m/500m CPU and 64/256 Mi; PostgreSQL 50m/500m and 128/256 Mi. Scaling is manual (`kubectl scale`); there is no metrics-server in the lab, so no HorizontalPodAutoscaler was used — E3 shows the two replicas used about 28–34 millicores each at 30 req/s, far from their limits. `enableServiceLinks: false` is set because the Service named `api` would otherwise inject `API_PORT=tcp://…`, which the API reads as its listen address.

4. Monitoring, Logging, Metrics and Alerting Design

4.1 Metrics in the service (feat/metrics patch)

The hoWallet API had no metrics endpoint. Commit `60cd5791` on branch `feat/metrics` (base `06aea177`; shipped as `patch/feat-metrics.diff`) adds an opt-in Prometheus endpoint: with `METRICS_ENABLED=true` a middleware is installed first in the chi router and serves `GET /metrics` ahead of the logger and the rate limits; with the default `false` neither the route nor the middleware exists, so production behaviour is unchanged. A dedicated registry is used (no global state), and an optional `METRICS_TOKEN` requires a bearer token for scraping. The route label is the chi pattern (e.g. `/api/accounts/{id}`), not the raw path, and unmatched requests are labelled `unmatched`, which keeps cardinality bounded. Ten tests cover it (7 middleware, 3 router); all 10 pass (evidence/06-metrics-tests.txt).

Table 7. Metrics exposed by the API (names fixed in the patch).

Metric	Type / labels	Used for
http_requests_total	counter {method, route, status}	request rate, error-rate alert, dashboard
http_request_duration_seconds	histogram {method, route}, buckets 5 ms … 10 s	p50/p95/p99, latency alert [6]
http_requests_in_flight	gauge	saturation (dashboard)
db_pool_acquired_conns / total / max	gauges from pgxpool.Stat	pool starvation (E1 diagnosis)
process_*, go_*	standard collectors	CPU per pod in E3
up (Prometheus)	per scraped pod	ApiReplicaDown alert

4.2 Structured logs and request correlation

Both tiers log one JSON object per request. nginx adopts an incoming `X-Request-Id` or mints `$request_id` and forwards it; the API's chi `RequestID` middleware adopts the header, echoes it as `X-Request-ID` and writes it into its request log line together with `user_id` and `household_id`; the same lines are also persisted to the `app_logs` table every 2 s. One identifier therefore joins the gateway line, the API line and the database copy (E5 found all 11 test requests in all three places).

```
# client-supplied id, evidence/04-correlation.txt
gateway {"time":"2026-11-04T11:22:28+00:00","request_id":"endterm-corr-A-001","method":"GET","uri":"/api/households",
         "status":200,"upstream_addr":"10.244.0.23:8080","request_time":0.015,"upstream_response_time":"0.014"}
api     {"time":"2026-11-04T11:22:28.606293453Z","level":"INFO","msg":"request","method":"GET","path":"/api/households",
         "status":200,"duration":14014531,"remote":"10.244.0.1","request_id":"endterm-corr-A-001","user_id":"e078777c-…"}
# no incoming id: nginx minted 715b152b146a75d71299d05c96c92b72 and the API logged the same id
```

Listing 2. One request seen in the gateway and API logs under the same correlation id.

4.3 Collection and dashboard

Prometheus scrapes every 2 s and evaluates rules every 2 s. It discovers API pods through `dns_sd_configs` on the headless Services, so `up` exists per replica, and scrapes the baseline under its own job `api-baseline` (Figure 2). The Grafana dashboard *hoWallet API — Endterm* (refresh 5 s) has seven panels: request rate by status, error rate %, in-flight requests, latency p50/p95/p99, replicas up per instance, firing alerts and DB pool connections (Figure 4, Figure 9, Figure 11, Figure 16).

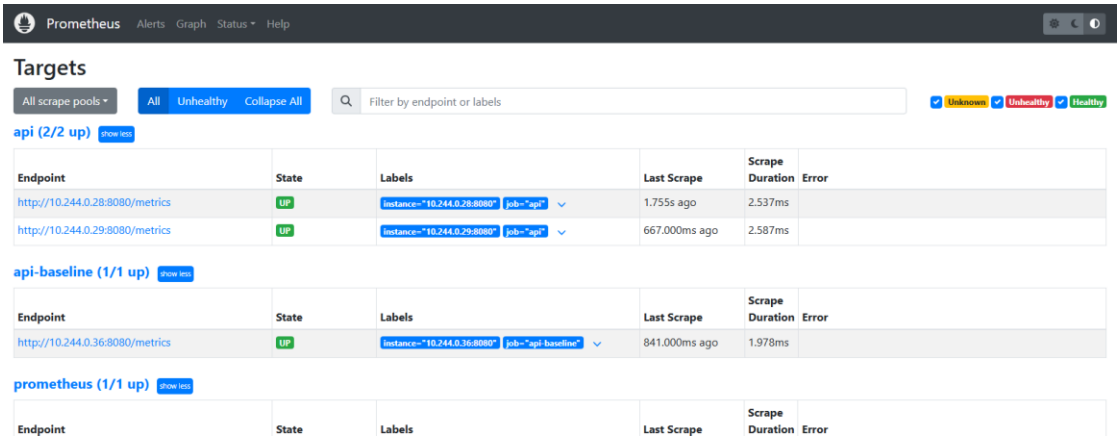


Figure 2. Prometheus targets: two api replicas discovered per pod, the baseline under its own job, all up (evidence/03-targets.txt).

4.4 Alert definitions

The three rules are in `k8s/50-prometheus.yaml` (evidence `05-alert-rules.txt`). Before the experiments the error and latency rules were scoped to `job="api"` and `route!="/health"`: unscoped, they mixed in baseline traffic and the cheap kubelet `/health` probes that dilute the error ratio. `ApiReplicaDown` was rewritten during the experiments after it was shown unable to fire (Section 8.5).

Table 8. Alert definitions, expected response and observed detection delay (from fault injection to state firing, AlertWatcher polling every 1 s).

Alert / severity	Detects	Threshold (expr, for)	Expected response	Observed delay
ApiHighErrorRate / critical	server-side failures (5xx) across the API	5xx / all > 0.05 over rate[30s], job=api, route≠/health; for 10 s	check DB and pod logs by request_id, recent deploys; gateway already returns JSON 503	15.2–20.2 s (E2, 4 runs); 16.3 / 17.3 s (E4c)
ApiHighLatencyP95 / warning	slow responses (DB locks, saturation)	histogram_quantile(0.95, rate[30s]) > 0.5 s; for 10 s	inspect DB locks, pool saturation, in-flight; scale replicas	18.2 / 19.3 s (E4d); 20.2–23.3 s as true positive in E2/E4c
ApiReplicaDown / critical (fixed rule)	fewer than 2 healthy, scrapeable API replicas	(count(up{job=api}==1) < 2) or absent(up{job=api}==1); for 6 s	Kubernetes restarts/reschedules; on-call confirms 2 Ready pods, reads pod events	10.2 / 11.2 s (E4b reps 2–3)
ApiReplicaDown (original)	same intent	up{job=api} == 0; for 6 s	–	never fired: pending 4.1 s, cleared 8.1 s (E4b rep 1); pending 5.1 s (E1 crash)

The delay has a floor set by the design: a 2 s scrape, a 30 s rate window that must accumulate enough bad samples to cross the threshold, the `for` duration (6–10 s) and a 2 s evaluation interval. The observed 10–23 s is consistent with that budget; shortening it would trade detection speed for false positives (Google SRE book, ch. 6 [11]).

4.5 How an incident is detected and diagnosed

- **Detect:** an alert goes pending, then firing (Prometheus /alerts, Grafana *Firing alerts*).
- **Localise:** the dashboard separates the cases — error rate up and replicas 2/2 points to a dependency (E2: 5xx with `failed to check household membership`); replicas < 2 points to a pod (E4b); latency up with 0 % errors points to contention (E4d: table locks).
- **Explain one request:** take the `request_id` from the client response or the gateway log and grep the API log / `app_logs` for it (Listing 2); the line carries status, duration, user and household.
- **Confirm the cause:** `kubectl get events` / pod `lastState` (E4b showed `Liveness probe failed … Killing`, the final check found postgres `OOMKilled`), DB pool gauges (E1 diagnosis).

5. Security Controls and Intrusion-Tolerance Strategy

The strategy is defence in depth with fail-safe defaults: an attacker who passes one layer meets the next, and a failing check refuses rather than allows. Controls map to OWASP API Security Top 10 (2023) risks [12]. Exact expected behaviour per control is in `notes/security-controls.md`.

Table 9. Implemented security controls and their verification.

Control	Implementation	OWASP API	Verified by
Authentication	JWT HS256, issuer + exp required, live-session check; 401 JSON	API2	E5 (4 x 401), middleware spec tests
Object/tenant authorisation	membership check per request → 404 (existence not revealed); queries scoped by household_id	API1	E5 (4 x 404, 1 x 400), API test over every route
Input validation	typed JSON decode, 1 MB decode limit, positive decimal amounts, UUID path ids	API8	E6 (4 x 400)
Body-size limit at the edge	nginx `client_max_body_size 2m` → JSON 413	API4	E6 (3 MB → 413)
Rate limiting	1200/min per IP, 10/min on /auth, 600/min per user, 60/min creates; 429 JSON + Retry-After	API4	E3 (baseline 429s), E6
Brute-force lockout	5 failures per (e-mail, IP) / 15 min → 429 Retry-After 900, even for the right password	API2	E6
Client-IP integrity	gateway overwrites X-Real-IP, clears True-Client-IP	API8	E6 (baseline spoofable, gateway not)
Duplicate/abuse protection	client_id idempotency, unique index	API6	E2, E7
Secrets handling	Kubernetes Secret via envFrom; API refuses to start with a short JWT_SECRET; refresh tokens hashed	API8	deployment
Audit logging	per-request JSON log with request_id/user_id; app_logs; audit_logs for writes	API9	E5 (11 app_logs rows)
Least privilege	non-root uid 65532, read-only rootfs, caps dropped, no SA token; NetworkPolicies	API8	manifests, 02-get-all.txt
Safe failure	membership check fails closed (500) when the DB is down	–	E2 error bodies

Intrusion tolerance. The design aims to keep the service correct while partly compromised or abused: brute force is slowed by limits and lockout but cannot exhaust capacity (E6 probe stayed 43/43 OK); a stolen token for one tenant gives no access to another (E5); a retried or replayed write cannot double a balance (E7); a compromised pod is network-confined. Known weaknesses are stated, not hidden: limits and lockout are per replica, session liveness fails *open* on a DB error (to avoid mass sign-out), there is no dedicated failed-login security event, and the API's database user owns every table.

6. Verification and Validation Plan

6.1 Automated tests

Verification uses the project's existing test suites; validation uses the lab experiments against the acceptance criteria R1–R8 (Table 3).

Table 10. Automated test inventory (notes/test-inventory.md).

Level	Scope	Count	Status in this project
Unit (Go)	services, handlers, middleware on gomock mocks	2095 top-level tests (3048 incl. subtests)	run on 4 Nov 2026 at 06aea177: exit 0, 0 failed, 51 s
Integration (Go, tag integration)	repositories on real PostgreSQL (testcontainers)	278 (static count)	not re-run (Docker busy with the lab)
API/system (tags integration api)	real app over a fresh DB per test	538 (static count)	not re-run
Metrics patch	label = route pattern, status, in-flight, scrape not counted, token, disabled = no route	10 (7 + 3)	run on 4 Nov 2026 at 60cd5791: 10 passed (evidence/06-metrics-tests.txt)
Web	pure logic, components (MSW), Playwright journeys	~901 / ~273 / 21 files	not re-run

Tests most relevant to fault tolerance include `TestTransactionService_ClientIDConcurrentBurstWritesOnce` (idempotency under a concurrent burst), `TestAuthzHousehold_EveryRouteRefusesWithoutAValidMembership` (tenant isolation on every route), `TestLogin_AfterFiveFailures_EvenTheRightPasswordIs429WithRetryAfter` and `TestCreate_ForeignCurrencyWithNoRate_Is422AndWritesNothing` (the E8 behaviour).

6.2 Validation: acceptance criteria and integrity checks

Each experiment has a pre-declared criterion (Table 20 repeats them with the verdict). Data integrity is checked in SQL, not through the API: for every test account, row count = acknowledged writes, `count(DISTINCT client_id)` = row count, `balance = −sum(amount)`, and globally no duplicate `(household_id, client_id)` pair. Tenant isolation is checked with a before/after snapshot of counts and MD5 of the victim's rows.

7. Fault-Injection and Security-Test Methodology

7.1 Environment and dates

All measured runs were executed on **4 November 2026**, between about 16:20 and 17:30 local time (UTC+05:00; log timestamps are partly UTC, e.g. 11:22Z cluster up, 12:22Z E8). The preceding days of the project period (28 October – 3 November) were used for design, the metrics patch, manifests and scripts; no measurement from those days is reported. Environment: one kind node `ftr-endterm-control-plane` (Kubernetes v1.33.1, containerd 2.1.1, WSL2 kernel 6.6.87.2) on a Windows 11 laptop; images `howallet-api:endterm` (both configurations), `nginx:1.27-alpine`, `postgres:16-alpine`, `prom/prometheus:v2.54.1`, `grafana/grafana:11.2.0`.

7.2 Tooling

- `scripts/ftlab.py` — **Probe**: open-loop load generator with a fixed offered rate and a thread pool, so a hanging request does not slow the probe (closed-loop probes under-report outages); **EndpointWatcher**: `kubectl get endpoints -w`, time-stamped on arrival; **AlertWatcher**: polls Prometheus `/api/v1/alerts` every second; `known_state()` waits for all workloads Ready and no alert active before each run.
- `scripts/faults.py` — E1 (pod delete, graceful or `--grace-period=0 --force`) and E2 (`scale postgres 0`, or SIGSTOP/SIGCONT of all postgres processes on the node).
- `scripts/exps.py` — E3–E8; `scripts/final_check.py` — final SQL integrity; `scripts/build_summary.py` — tables from JSON; `scripts/render_evidence.py` — terminal renders of the text logs.
- Workload: `GET /api/accounts` with a token and `X-Household-ID` (membership check + accounts query, so it needs the database). Client timeout 6 s (E1, E3) or 12 s (E2). Synthetic users created by `scripts/setup_users.py` (register → mark e-mail verified in SQL → log in).

7.3 Measurement definitions

Table 11. Definitions of the reported measurements.

Measurement	Definition
Success / failure	2xx = success; anything else, connection error or client timeout = failure. 429 in E3 is reported separately as the limiter working.
Error rate, availability	failed / all and 2xx / all requests in the stated window
Latency	client-side, through the host NodePort; average and p95 of all requests in the window
Detection time	injection → first signal: endpoint removal (Kubernetes), first failed request, or alert state firing
Recovery time	E1: injection → replacement pod back in endpoints (or end of last failure if later); E2: restore command → end of the last failed request
User-visible outage	first failed request → end of the last failed request
E1 window	injection → recovery + 15 s; E2 window: injection → restore ("during outage") and the whole run ("interval")

7.4 Experiment catalogue

Table 12. Experiments: type, injection, hypothesis and repetitions (6 fault/abnormal, 2 security/misuse; E6 also includes abuse).

ID	Type	Injection / steps	Hypothesis	Reps
E1	fault	delete one API pod under an 8 req/s probe (graceful and forced)	endterm: 0 failures; baseline: outage until the replacement is ready	3 + 3
E2	fault	PostgreSQL stopped (scale 0) or frozen (SIGSTOP) for 20 s; both configs probed 6 req/s each + a writer retrying with the same client_id	fast JSON errors, alert fires, no lost or duplicate data	2 stop + 2 freeze per config
E3	abnormal load	5 → 30 → 5 req/s (30 s / 60 s / 30 s), 5 users	≥ 99 % success, p95 < 500 ms	2 + 2
E4	monitoring	a) 120 s quiet; b) SIGSTOP one API process; c) DB scaled 0 for 25 s; d) 22 x `LOCK TABLE accounts` held 1.5 s	each rule fires on its scenario only	1 + 3 + 2 + 2
E5	security	11 requests: no/garbage/forged token, user B against tenant A	401/404, data unchanged, audit trail	1
E6	misuse	malformed JSON, negative amount, bad UUID, 3 MB / 1.5 MB bodies, 12 wrong passwords, spoofed X-Real-IP	400/413/429, probe unaffected	1
E7	fault + integrity	200 creates with unique client_id, one API pod deleted at t+16.7 s, client retries with the same id	200 rows, 0 duplicates	2 + 2
E8	dependency failure	RATES_BASE_URL → unresolvable host (rolling update)	KZT 201, USD/EUR 422 no_rate, nothing wrong written	1 (4 requests)

Restoring a known state. Every run starts with `known_state()`; E2 waits until both probes have been all-200 for 15 s; E8 restores the environment variable with a second rolling update.

Invalid runs. E2 freeze reps 3 and 4 **did not inject a fault**: the command went through Windows `cmd.exe`, which passed the quotes in `pkill -f '^postgres'` literally, so no process matched (0 failures, no alerts — a correct negative for the monitoring, but not an experiment). Both JSON files were renamed `*-INVALID-not-injected.json`, kept in the log with a note, excluded from all results, and the freeze variant was re-run as reps 5 and 6 with `pkill -STOP -x postgres`.

8. Results and Baseline-versus-Endterm Comparison

Per-run numbers below are read from `evidence/results/*.json` when this report is built; the readable logs are `evidence/*.txt`. Latencies are in milliseconds, times in seconds after the injection.

8.1 Summary of measurements

Table 13. Required measurements, endterm versus baseline (ranges over the valid repetitions).

Experiment	Config	Requests: failed / total	Error rate	Avg / p95 latency	Detection	Recovery	Availability
E1 replica loss (graceful, 2 reps)	endterm	0 / 139, 0 / 140	0 %	24 / 44; 133 / 1138	endpoint drop 0.2 s; no alert	1.9 / 1.6 s	100 %
	baseline	137 / 139, 138 / 139	98.6–99.3 %	5839–5880 / 6017–6028	endpoint drop 0.2 s; no alert	23.3 / 23.4 s	1.4 / 0.7 %
E1 forced kill (1 rep)	endterm	0 / 131	0 %	20.5 / 38.5	0.2 s; ApiReplicaDown pending only	0.8 s	100 %
	baseline	9 / 140	6.4 %	179 / 1573	0.2 s	2.3 s	93.6 %
E2 DB outage 20 s (4 valid reps), during outage	endterm	118–121 / 121–122	97.5–99.2 %	1553–2128 / 5010–5022	ApiHighErrorRate firing 15.2–20.2 s	6.2 / 7.2 s (stop), 16.7 / 13.1 s (freeze)	whole run 45.5–65.3 %
	baseline	62–72 / 113–114	54.4–63.7 %	8277–10756 / 12015–12018 (client timeout)	no alert (not covered)	2.3 / 2.4 s (stop), 13.4 / 7.5 s (freeze)	whole run 78.2–84.3 %
E3 30 req/s step (2 reps)	endterm	0 / 1800 (x2)	0 %	10.6–11.1 / 26.2–27.9	no alert (correct)	–	100 %
	baseline	321 / 1800, 248 / 1800 (incl. 307 / 248 x	17.8 / 13.8 %	111.8 / 995.1; 10.0 / 26.5	no alert (not covered)	–	82.2 / 86.2 %

Experiment	Config	Requests: failed / total	Error rate	Avg / p95 latency	Detection	Recovery	Availability
		429)					
E4b hung replica (fixed rule, 2 reps)	endterm	13 / 404, 9 / 363	3.2 / 2.5 %	176 / 63; 137 / 32 (max 5.0 s)	ApiReplicaDown firing 10.2 / 11.2 s	back in endpoints 51.9 / 45.1 s (liveness restart)	96.8 / 97.5 %
E7 pod kill during 200 writes (2 reps)	endterm	0 / 200 (x2)	0 %	62 / 206; 48 / 150	endpoint drop 0.2 s	1.1 / 1.1 s (endpoints back)	100 %; 200 rows, 0 dup
	baseline	7 / 207 (x2), all retried	3.4 %	82 / 166; 80 / 187	endpoint drop 0.2 s	1.6 / 1.7 s (endpoints back)	96.6 % of attempts; 200 rows, 0 dup

E3 baseline "failed" counts include the 429 refusals (307 and 248) plus 14 connection errors in rep 1; E7 recovery is the endpoint drop/return pair from Table 19.

8.2 E1 — Replica failure

Table 14. E1 per run (window: injection → recovery + 15 s; probe 8 req/s).

Config	Rep	Req	Failed	Err %	Avg	p95	Ep. drop	Recovery	Outage	Avail %	Alerts
endterm	1 graceful	139	0	0.0	24.0	44.3	0.2	1.9	0.0	100.0	none
endterm	2 graceful	140	0	0.0	133.0	1137.8	0.2	1.6	0.0	100.0	none
endterm	3 crash	131	0	0.0	20.5	38.5	0.2	0.8	0.0	100.0	ApiReplicaDown pending 5.1 s, not fired
baseline	1 graceful	139	137	98.6	5838.6	6027.9	0.2	23.3	23.0	1.4	none
baseline	2 graceful	139	138	99.3	5880.0	6016.7	0.2	23.4	23.1	0.7	none
baseline	3 crash	140	9	6.4	179.2	1573.1	0.2	2.3	2.0	93.6	none

Endterm. No request failed in any of the three runs. The deleted pod left the endpoints after 0.2 s and the replacement was in rotation after 1.9, 1.6 and 0.8 s. Rep 2 shows the cost of hiding the failure: p95 rose to 1138 ms (max 1836 ms) because some requests reached the terminating pod and were retried on the other replica by the gateway. **Baseline.** A graceful delete caused a 23.0 / 23.1 s user-visible outage (137 and 138 of 139 requests failed, mostly 6 s client timeouts), while the forced kill caused only 2.0 s (9 failures).

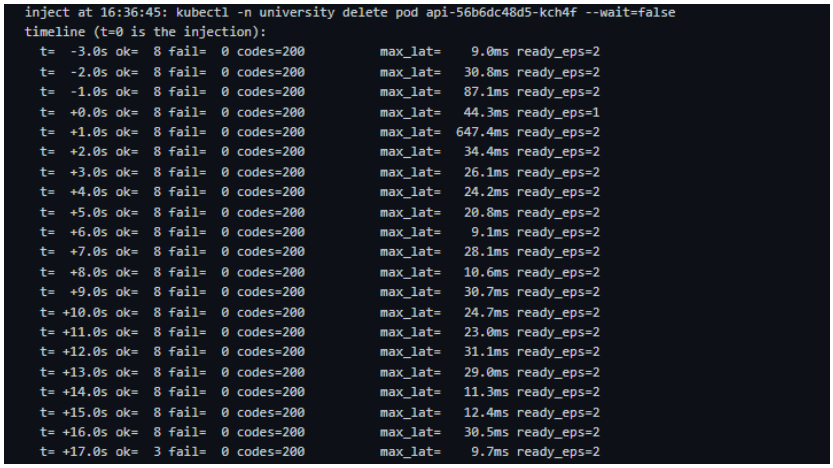


Figure 3. E1 endterm rep 1 (graceful delete): ready endpoints drop to 1 and return to 2, every 1-s bucket 8/8 OK (evidence/e1-replica-failure.txt).

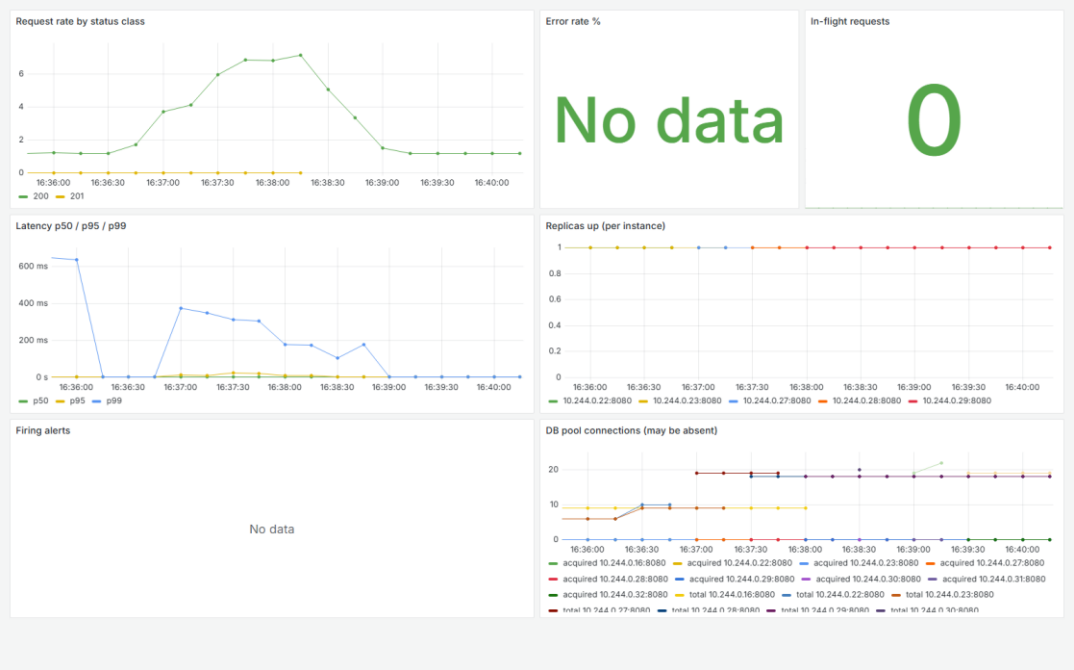


Figure 4. Grafana during the E1 endterm runs: only 200/201 responses, error rate has no 5xx data, new replica IPs appear in Replicas up as pods are replaced; p99 rises to about 370 ms.

```
inject at 16:38:12: kubectl -n university delete pod api-baseline-69bbd8cf45-q6dt9 --wait=false
timeline (t=0 is the injection):
  t=  -3.0s ok=  8 fail=  0 codes=200          max_lat=   27.8ms ready_eps=1
  t=  -2.0s ok=  8 fail=  0 codes=200          max_lat=   16.4ms ready_eps=1
  t=  -1.0s ok=  8 fail=  0 codes=200          max_lat=   25.7ms ready_eps=1
  t=  +0.0s ok=  2 fail=  6 codes=200,ERR      max_lat= 6030.8ms ready_eps=0
  t=  +1.0s ok=  0 fail=  8 codes=ERR          max_lat= 6030.7ms ready_eps=1
  t=  +2.0s ok=  0 fail=  8 codes=ERR          max_lat= 6011.6ms ready_eps=1
  t=  +3.0s ok=  0 fail=  8 codes=ERR          max_lat= 6011.5ms ready_eps=1
  t=  +4.0s ok=  0 fail=  8 codes=ERR          max_lat= 6008.3ms ready_eps=1
  t=  +5.0s ok=  0 fail=  8 codes=ERR          max_lat= 6034.8ms ready_eps=1
  t=  +6.0s ok=  0 fail=  8 codes=ERR          max_lat= 6020.2ms ready_eps=1
  t=  +7.0s ok=  0 fail=  9 codes=ERR          max_lat= 6016.7ms ready_eps=1
  t=  +8.0s ok=  0 fail=  8 codes=ERR          max_lat= 6016.1ms ready_eps=1
  t=  +9.0s ok=  0 fail=  8 codes=ERR          max_lat= 6026.7ms ready_eps=1
  t= +10.0s ok=  0 fail=  8 codes=ERR          max_lat= 6033.7ms ready_eps=1
  t= +11.0s ok=  0 fail=  8 codes=ERR          max_lat= 6015.8ms ready_eps=1
  t= +12.0s ok=  0 fail=  7 codes=ERR          max_lat= 6013.4ms ready_eps=1
  t= +13.0s ok=  0 fail=  8 codes=ERR          max_lat= 6009.6ms ready_eps=1
  t= +14.0s ok=  0 fail=  8 codes=ERR          max_lat= 6029.3ms ready_eps=1
  t= +15.0s ok=  0 fail=  9 codes=ERR          max_lat= 6027.9ms ready_eps=1
  t= +16.0s ok=  0 fail=  7 codes=ERR          max_lat= 6025.5ms ready_eps=1
  t= +17.0s ok=  0 fail=  3 codes=ERR          max_lat= 6008.7ms ready_eps=1
```

Figure 5. E1 baseline rep 1: every request fails for 23 s although the replacement is Ready (ready_eps=1) after about 1.7 s.

Root-cause hypothesis for the 23 s baseline outage (not proven). `/health` does not touch the database, so the new pod is Ready as soon as the process listens. In the diagnostic runs (`e1-baseline-diagnostic.txt`, Figure 6) the replacement answered a 1 req/s probe within 2.3 s, but under the queued 8 req/s burst its pool showed `total=40=max`, `acquired=33`, `in_flight=34`, and DB-backed requests hung until about t+20–25 s — the moment the startup `exchange_rates` job gave up on the external provider (522 after ~20 s). Pool starvation during startup jobs is the likely mechanism; why the forced kill did not show it was not established. The endterm masked it: retries to the healthy replica and the 5 s gateway cap.

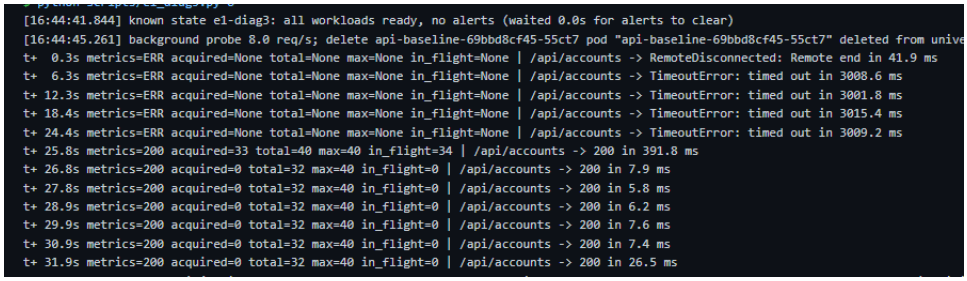


Figure 6. E1 baseline diagnostic at 8 req/s: metrics unreachable/timeouts until t+24.4 s, then the pool is full (total=40=max, in_flight=34); in the log the rates job fails at about the same moment.

8.3 E2 — Database outage (dependency failure)

Table 15. E2 per run, both configurations probed at the same time (during-outage window; "interval" = whole run; writes = acknowledged / rows / distinct client_id / balance; replays answered 200).

Run	Config	Req	Fail	Err %	Avg	p95	Codes	Alert firing	Recov.	Interval avail %	Writes; replays
rep1-stop	endterm	122	119	97.5	1553.4	5010.4	200:3, 500:19, 503:100	16.2 s	6.2	49.8	35/35/35/ok; 1
rep1-stop	baseline	114	62	54.4	8277.2	12015.1	200:52, 500:13, timeout:49	no rule	2.3	79.2	34/34/34/ok; 1
rep2-stop	endterm	121	118	97.5	1558.2	5017.7	200:3, 500:17, 503:101	15.2 s	7.2	45.5	34/34/34/ok; 0
rep2-stop	baseline	113	63	55.8	8638.8	12016.1	200:50, 500:12, timeout:51	no rule	2.4	78.2	34/34/34/ok; 1
rep5-freeze	endterm	122	121	99.2	2127.9	5021.9	200:1, 503:115, 500:6	20.2 s	16.7	65.3	55/55/55/ok; 1
rep5-freeze	baseline	113	72	63.7	10756.1	12017.7	200:41, timeout:70, 500:2	no rule	13.4	84.3	41/41/41/ok; 2
rep6-freeze	endterm	121	120	99.2	2033.0	5012.1	200:1, 503:119, 500:1	20.2 s	13.1	49.9	39/39/39/ok; 2
rep6-freeze	baseline	113	68	60.2	10549.0	12016.6	200:45, timeout:68	no rule	7.5	79.6	38/38/38/ok; 2

Endterm. Clients received a JSON 503 from the gateway (`service_unavailable`, `Retry-After: 2`) or the API's JSON 500 (`failed to check household membership`); p95 during the outage was capped at 5.0 s by the gateway. ApiHighErrorRate fired at 15.2–20.2 s; ApiHighLatencyP95 also fired (21.3–22.3 s stop, 20.2 s freeze), correctly, since p95 really was 5 s. **Baseline.** Requests hung until the 12 s client timeout (49–70 timeouts per run, p95 12.0 s); no alert covers it. **Integrity:** in all 8 configuration-runs every acknowledged write was stored exactly once and `balance = −sum`; 0–2 retries per run were answered 200, i.e. the server recognised the `client_id` and replayed the stored row instead of writing a second one.

The honest trade-off. The endterm did not improve availability here; it made failure fast and visible. Over the whole run the baseline served more requests successfully (78.2–84.3 % vs 45.5–65.3 %) and recovered sooner after the restore (2.3–13.4 s vs 6.2–16.7 s): its hung requests simply completed once the database returned, whereas the gateway had already answered 503 and, with `max_fails=1`, kept marking upstreams failed while 5 s requests were still timing out.

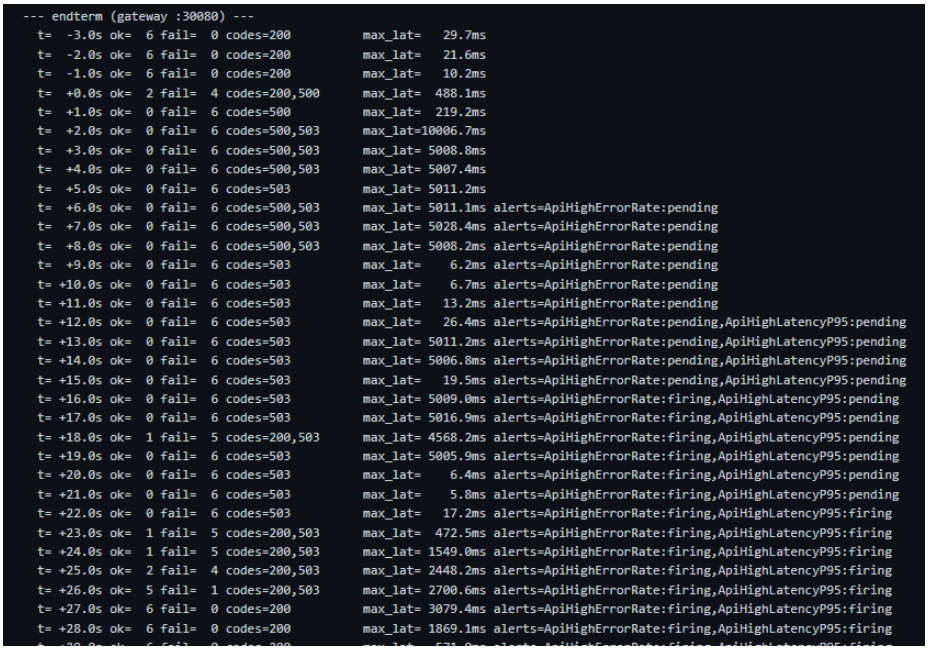


Figure 7. E2 rep 1 (stop), endterm: 500/503 with the latency capped near 5 s, ApiHighErrorRate pending at +6 s and firing at +16 s, successes return from +23 s.

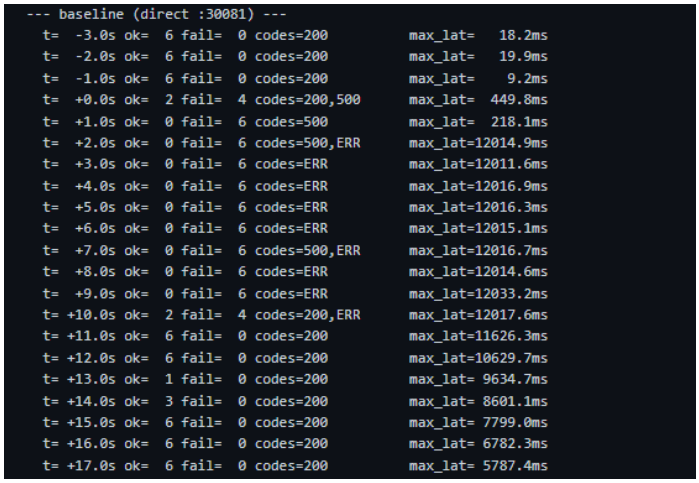


Figure 8. E2 rep 1 (stop), baseline at the same time: requests hang until the 12 s client timeout (codes=ERR), then all requests complete once the DB is back.

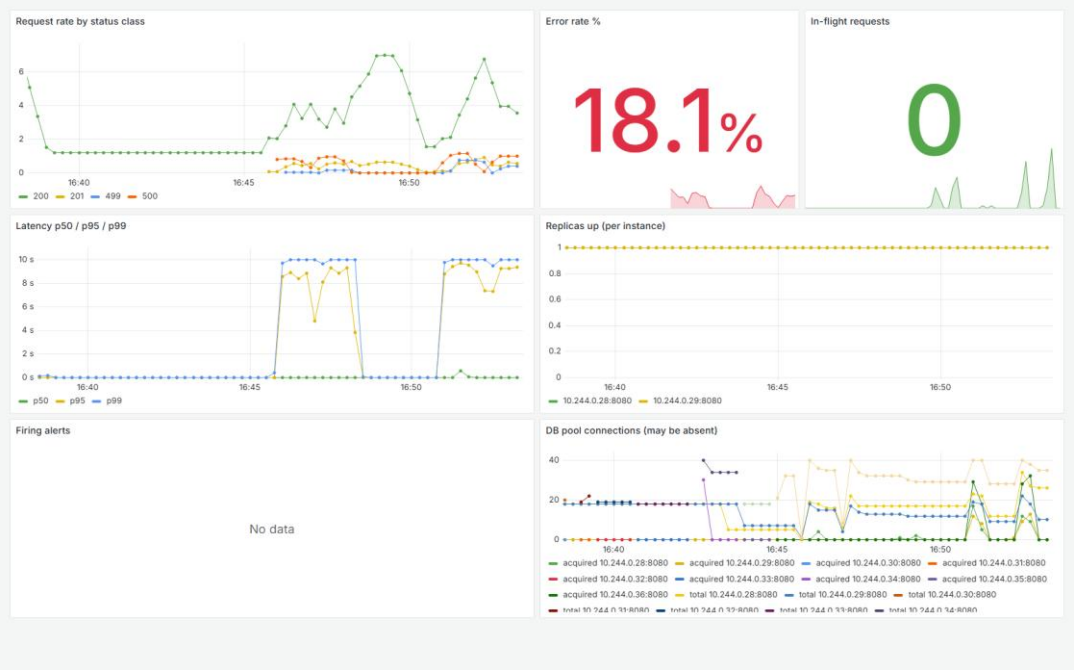


Figure 9. Grafana after the E2 series: latency p95/p99 at the 5–10 s ceiling during each outage, 500 responses, DB pool connections dropping and returning.

8.4 E3 — Load increase

Table 16. E3, 60 s step at 30 req/s ("failed" = connection errors; 429 counted separately; CPU in millicores from process_cpu_seconds_total).

Config	Rep	Req	OK	Failed	429	OK %	OK req/s	Avg	p95	API CPU (per pod)	PG CPU	Alerts
endterm	1	1800	1800	0	0	100.0	30.0	10.6	26.2	33.6, 27.9	61.3	none
endterm	2	1800	1800	0	0	100.0	30.0	11.1	27.9	31.7, 32.7	43.4	none
baseline	1	1800	1479	14	307	82.2	24.65	111.8	995.1	50.6	64.7	none
baseline	2	1800	1552	0	248	86.2	25.87	10.0	26.5	64.3	54.5	none

The endterm served 1800/1800 requests at the full 30 req/s with p95 26–28 ms; each API pod used about 28–34 m CPU and each gateway pod 10–12 m. The baseline was throttled by its own limiter: one replica counts all traffic against the 1200/min per-IP limit (all clients share one SNAT address), so 307 and 248 requests were refused with 429 (`rate_limited`). The endterm passed only because the limit is counted per replica — this is the per-replica-limit limitation of Section 9, not extra capacity.

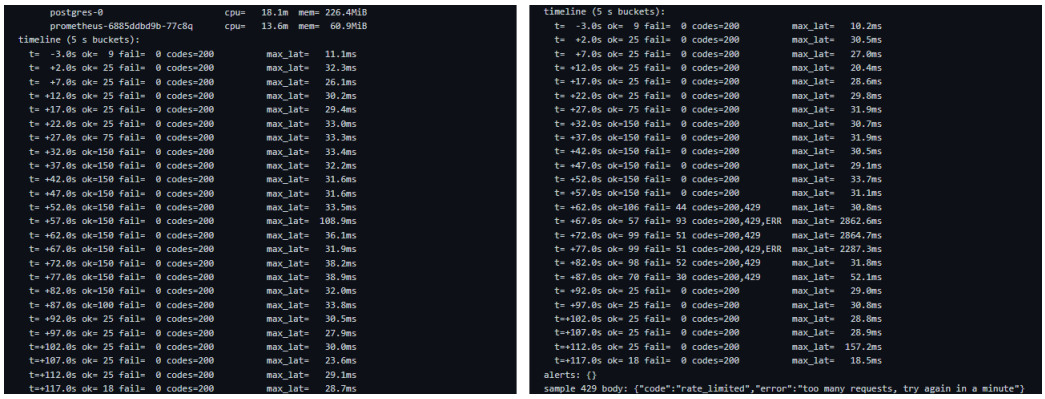


Figure 10. E3 timelines in 5-s buckets: endterm rep 1 (left) all 150/150 at 30 req/s; baseline rep 1 (right) refuses requests with 429 from t+62 s.

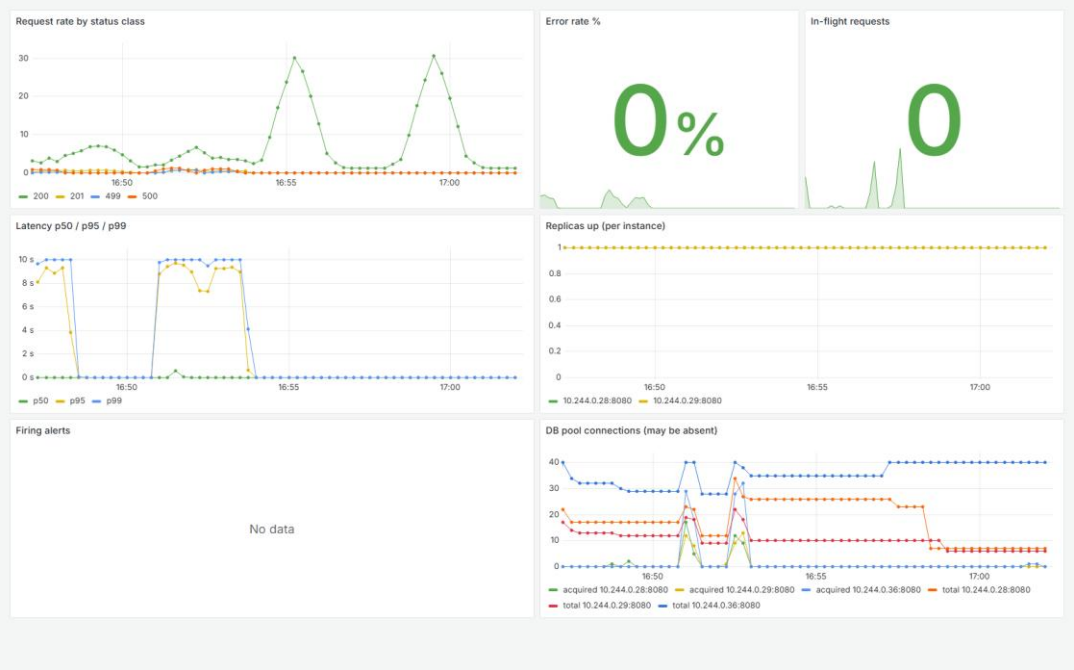


Figure 11. Grafana after E3: the request rate peaks at about 30 req/s during the E3 steps; the earlier 10 s latency plateaus belong to the preceding E2 outages; no firing alert.

8.5 E4 — Monitoring and alert tests

Table 17. E4 per run: probe during the fault window and alert timings (s after injection).

Run	Expected alert	Req	Fail	Err %	Avg	p95	Max	Alert timings	Fired
quiet-window	none	601	0	0.0	11.4	25.2	182.3	120 polls, nothing pending	none
replica-down-rep1	ApiReplicaDown (original rule)	392	13	3.3	178.0	31.4	5026.7	ApiReplicaDown: pend 4.1, fire None, clear 8.1	none
replica-down-rep2	ApiReplicaDown	404	13	3.2	175.8	63.0	5030.2	ApiReplicaDown: pend 4.1, fire 10.2, clear 60.7	ApiReplicaDown
replica-down-rep3	ApiReplicaDown	363	9	2.5	136.6	31.8	5032.5	ApiReplicaDown: pend 6.1, fire 11.2, clear 53.6	ApiReplicaDown
error-rate-rep1	ApiHighErrorRate	152	148	97.4	1721.5	5010.6	10030.7	ApiHighErrorRate: pend 7.2, fire 17.3, clear 55.6; ApiHighLatencyP95: pend 13.2, fire 23.3, clear None	ApiHighErrorRate, ApiHighLatencyP95
error-rate-rep2	ApiHighErrorRate	152	146	96.0	2145.8	5028.1	10013.7	ApiHighErrorRate: pend 6.2, fire 16.3, clear 58.6; ApiHighLatencyP95: pend 12.2, fire 22.3, clear 58.6	ApiHighErrorRate, ApiHighLatencyP95
latency-rep1	ApiHighLatencyP95	240	0	0.0	643.2	1420.8	1586.6	ApiHighLatencyP95: pend 9.2, fire 19.3, clear 70.7	ApiHighLatencyP95
latency-rep2	ApiHighLatencyP95	240	0	0.0	640.1	1423.0	1537.6	ApiHighLatencyP95: pend 8.1, fire 18.2, clear 71.6	ApiHighLatencyP95

False negative found and fixed (E4b). The original rule `up{job="api"} == 0` could not fire in this deployment: a dead or hung pod leaves the headless-Service DNS within about 2 s, so its scrape target disappears and the series goes stale instead of staying at 0 for the 6 s `for`. In rep 1 it went pending at 4.1 s and cleared at 8.1 s while a replica stayed hung for about 47 s until the liveness probe restarted it (Figure 12). The rule was changed to count healthy replicas (`(count(up{job="api"}==1) < 2) or absent(up{job="api"}==1)`), reloaded, and re-tested: firing at 10.2 and 11.2 s, cleared after the restart. During the hung-replica window the gateway answered 13 / 13 / 9 JSON 503s (max 5.0 s) out of 363–404 requests. Because of `for: 6s`, a pod replaced within 2 s (E1) still does not page — intended.

```
rule: up{job="api"} == 0
  t=  -3.0s ok=  6 fail=  0 codes=200          max_lat=   31.2ms ready_eps=2
  t=  -2.0s ok=  6 fail=  0 codes=200          max_lat=   15.8ms ready_eps=2
  t=  -1.0s ok=  6 fail=  0 codes=200          max_lat=   29.0ms ready_eps=2
  t=  +0.0s ok=  4 fail=  2 codes=200,503      max_lat= 5006.9ms ready_eps=2
  t=  +1.0s ok=  2 fail=  3 codes=200,503      max_lat= 5026.7ms ready_eps=2
  t=  +2.0s ok=  4 fail=  3 codes=200,503      max_lat= 5008.4ms ready_eps=1
  t=  +3.0s ok=  2 fail=  3 codes=200,503      max_lat= 5012.1ms ready_eps=1
  t=  +4.0s ok=  5 fail=  2 codes=200,503      max_lat= 5009.5ms ready_eps=1 alerts=ApiReplicaDown:pending
  t=  +5.0s ok=  6 fail=  0 codes=200          max_lat=   57.8ms ready_eps=1 alerts=ApiReplicaDown:pending
  t=  +6.0s ok=  6 fail=  0 codes=200          max_lat=    9.5ms ready_eps=1 alerts=ApiReplicaDown:pending
  t=  +7.0s ok=  6 fail=  0 codes=200          max_lat=   21.7ms ready_eps=1 alerts=ApiReplicaDown:pending
  t=  +8.0s ok=  6 fail=  0 codes=200          max_lat=   13.1ms ready_eps=1
  t=  +9.0s ok=  6 fail=  0 codes=200          max_lat=   25.2ms ready_eps=1
```

Figure 12. E4b rep 1 with the original rule: ApiReplicaDown only pending (t+4…7 s) and clears while ready_eps stays 1 (the hung pod is restarted by liveness about 47 s later).

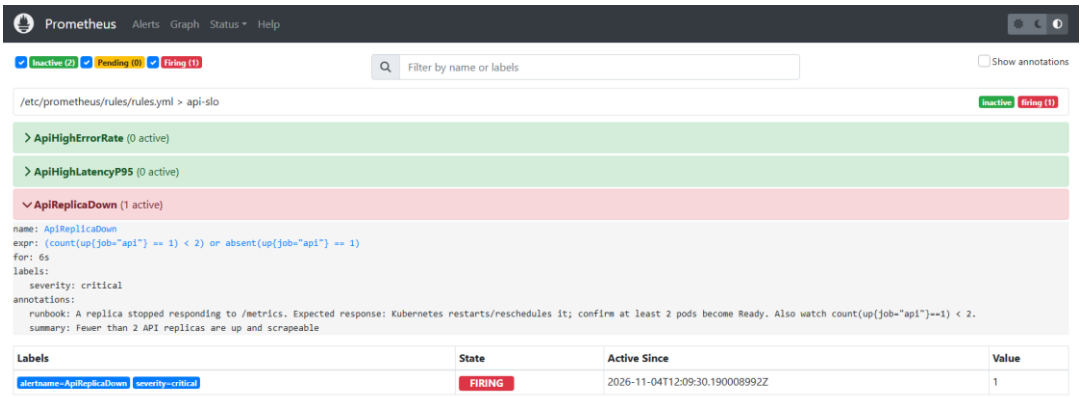


Figure 13. Prometheus: the fixed ApiReplicaDown rule firing in E4b rep 2.

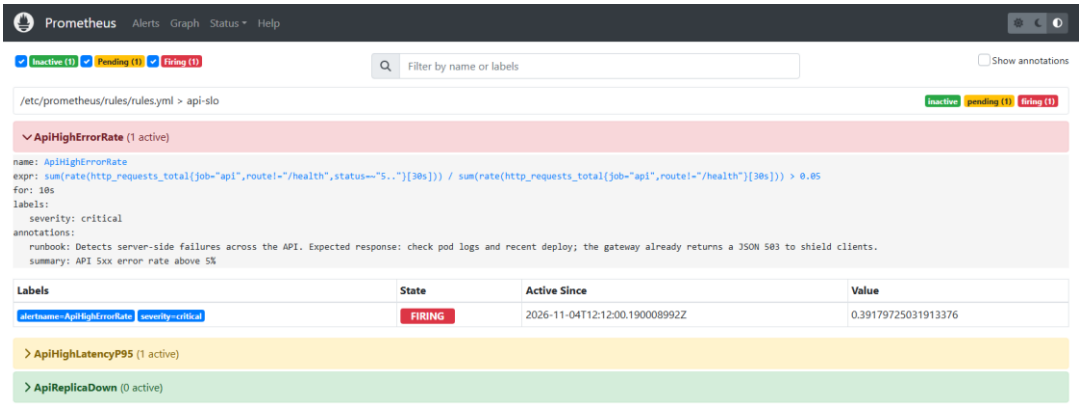


Figure 14. Prometheus: ApiHighErrorRate firing during the E4c database outage.

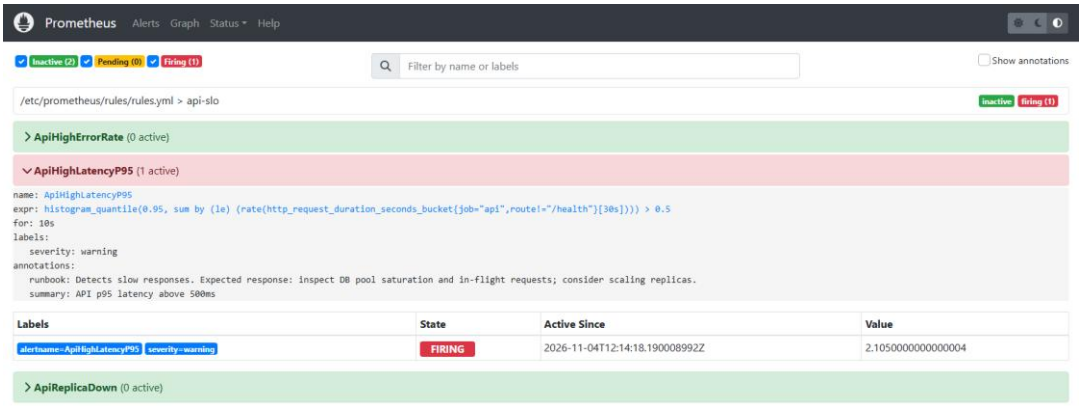


Figure 15. Prometheus: ApiHighLatencyP95 firing under the E4d table-lock bursts (0 failed requests, p95 1.42 s).

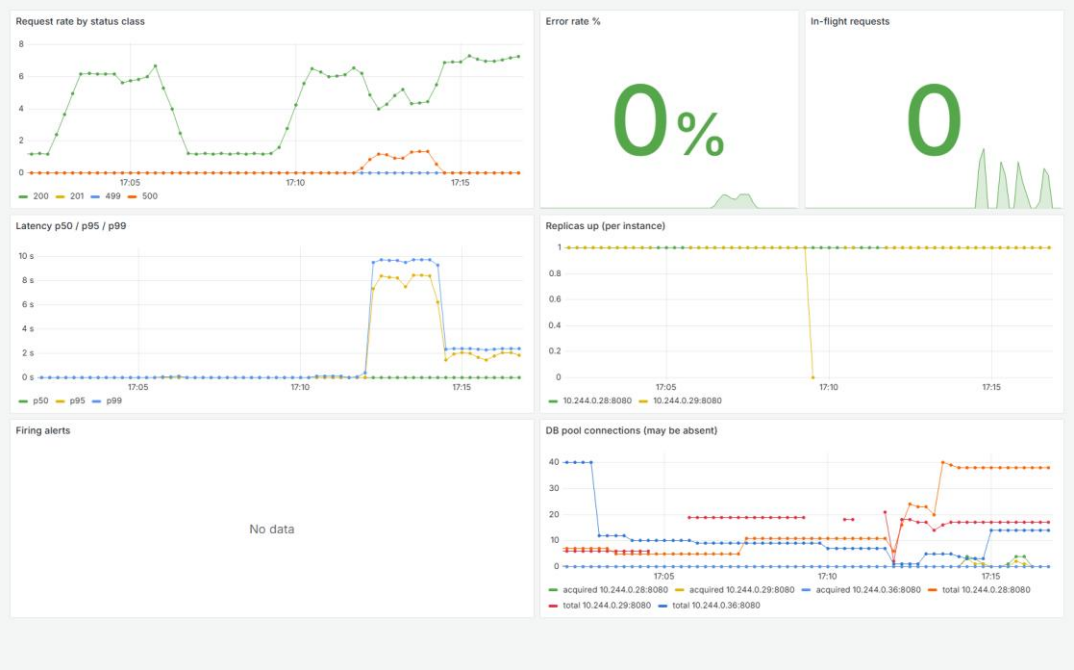


Figure 16. Grafana over the E4 series: one replica's up drops to 0 (E4b), 500s during E4c, p95/p99 near 8–10 s then about 2 s during E4d.

Table 18. Alerts triggered and their correctness across the measured runs.

Scenario (runs)	Alerts firing	Correct?	Comment
E4a quiet window (1)	0	yes (true negative)	120 polls, 601/601 OK
E3 load step, endterm (2)	0	yes	p95 < 30 ms, no 5xx
E2 DB outage (4 valid)	8 (4 x ErrorRate, 4 x LatencyP95)	yes	p95 really 5 s → latency alert is a true positive
E2 invalid reps 3–4 (2)	0	yes (nothing injected)	excluded from results
E4b original rule (1) / E1 (6)	0 (pending only)	no — false negative	rule replaced
E4b fixed rule (2)	2 x ApiReplicaDown	yes	cleared after liveness restart
E4c DB scaled 0 (2)	4 (2 x ErrorRate, 2 x LatencyP95)	yes	the script's strict flag says "correct=False" because it expected one alert only; both alerts reflected real symptoms
E4d table locks (2)	2 x LatencyP95	yes	error-rate alert correctly silent

In total 16 alert firings were recorded, none of them false positives; the only false negative was the original ApiReplicaDown rule. The baseline has no alert coverage at all (scraped, but not alerted on).

8.6 E5 and E6 — Security and misuse tests

E5 access control. Four requests without a valid token (missing, garbage, signed with a wrong secret) got 401. User B, holding a valid token, sent tenant A's household id to read accounts, read transactions, create an expense and delete an account: all four got **404** `household not found`, so A's existence is not even confirmed. Using B's own household with A's account id gave 400 `account not found`. The SQL snapshot of A (1 account, 3 transactions, MD5 of rows) was identical before and after (`data_unchanged = True`), and each request was found in the gateway log, the API log and `app_logs` (11 rows) under its request id. One case was mis-specified: `GET /api/households/{id}` returned 405 because that route does not exist — it is not an authorisation case, which is why the script's automatic pass flag is false.



Figure 17. E5: 401 for missing/invalid tokens, 404 for a foreign tenant, identical SQL snapshots before and after.

E6 input and limits. Malformed JSON (x2), a negative amount and a bad UUID → 400; a 3 MB body → JSON 413 from the gateway; 1.5 MB → 400 from the API's own 1 MB decode limit; no account was created. Twelve wrong passwords: 401 for attempts 1–9, **429 at attempt 10** (`Retry-After: 900`), 401 at 11 (the other replica, which had not yet seen 5 failures), 429 at 12, and the correct password afterwards was still 429 — the lockout works, but with two in-memory counters it needed 10 attempts instead of 5. A background probe stayed 43/43 OK (avg 13.2 ms, p95 37.9 ms). **Finding:** the baseline trusts a client-supplied `X-Real-IP` — the API logged `remote=203.0.113.77`, an address chosen by the client and therefore a fresh rate-limit and lockout key per request; through the gateway it logged `10.244.0.1`.

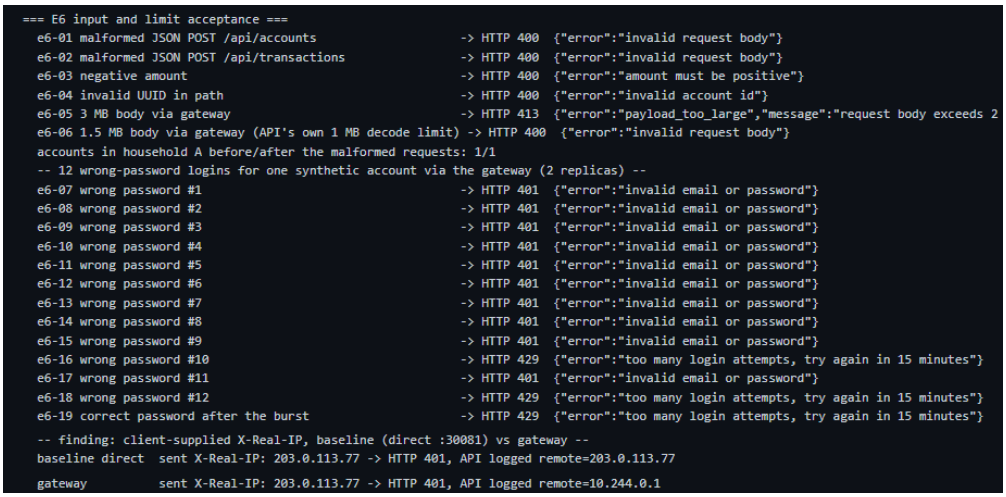


Figure 18. E6: validation, JSON 413, lockout sequence across two replicas, and the X-Real-IP comparison (baseline vs gateway).

8.7 E7 and E8 — Recovery, consistency and graceful degradation

Table 19. E7: 200 logical creates with unique client_id, one API pod deleted at t+16.7 s.

Config	Rep	HTTP attempts	Failed attempts	First attempt failed	Rows	Duplicates	Balances ok	Avg	p95	Endpoint drop / back (s)
endterm	1	200	0	0	200	0	yes	62.1	206.4	0.25 / 1.14
endterm	2	200	0	0	200	0	yes	48.3	149.5	0.23 / 1.12
baseline	1	207	7	5	200	0	yes	82.2	166.1	0.23 / 1.55
baseline	2	207	7	5	200	0	yes	79.5	187.2	0.22 / 1.69

Both configurations ended with exactly 200 rows, 0 duplicates and correct balances. The endterm needed no retry at all (the gateway and preStop hid the kill); the baseline lost 7 HTTP attempts per run to connection errors and 5 creates needed a client retry, which the `client_id` key made safe. E7 therefore did not exercise the server-side replay path on the endterm; that path was exercised in E2.

E8. With `RATES_BASE_URL` pointing at an unresolvable host and an empty rates archive, KZT expenses returned 201 (twice), USD and EUR returned **422** `no_rate`, the household gained exactly 2 rows and 0 non-KZT rows, and the

exchange_rates job logged ERROR. The environment was restored afterwards. Because the archive was already empty (the provider had answered 522 from inside the lab), E8 shows the no-rate path, not the stale-rate path.

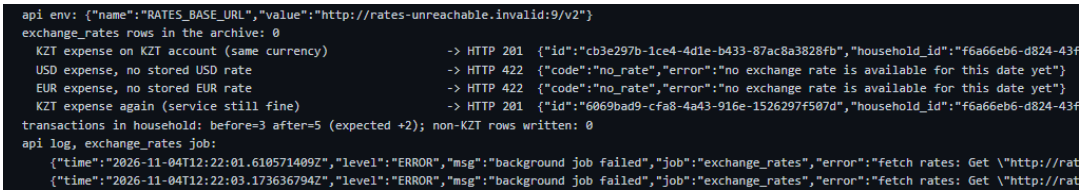


Figure 19. E8: base-currency writes succeed, foreign-currency writes are refused with 422 no_rate, nothing wrong is written, and the failing job is visible in the log.

8.8 Final integrity check

After all runs, `scripts/final_check.py` re-checked 35 accounts: `balance = −sum(amount)` everywhere and 0 duplicate `(household_id, client_id)` pairs in the whole database. The same check revealed that PostgreSQL had been **OOM-killed once** (exit 137, limit 256 Mi) at 12:22:04Z during the E8 rolling update, when up to four API pools plus the baseline were connected; it restarted by itself and no data was lost.

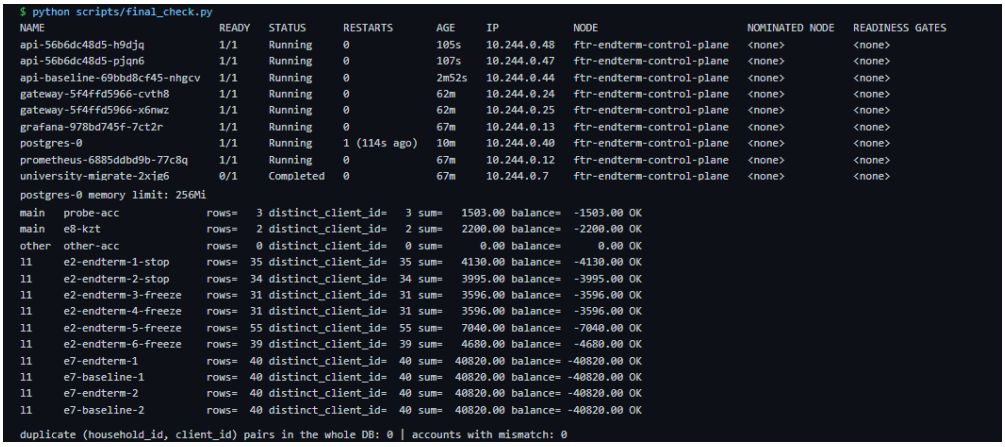


Figure 20. Final integrity (excerpt): pod states (postgres-0 restarted once; its lastState OOMKilled line is in the log), the first accounts checked, and the global duplicate check.

8.9 Pass/fail summary

Table 20. Pass/fail outcomes against the acceptance criteria.

Exp.	Acceptance criterion	Endterm result	Endterm	Baseline result	Baseline
E1	0 failed requests, recovery < 10 s	0 failed (3 runs), 0.8–1.9 s	PASS	137–138 / 139 failed, 23 s	FAIL
E2	fast JSON error, alert, recovery, no lost/duplicate data	JSON 503/500 ≤ 5 s, alert 15–20 s, integrity 4/4	PASS	12 s hangs, no alert; integrity 4/4	partial (integrity only)
E3	≥ 99 % at 30 req/s, p95 < 500 ms, no alert	100 %, p95 ≤ 28 ms	PASS	82–86 % (429)	FAIL
E4a	no alert in quiet window	0 alerts	PASS	–	–
E4b	ApiReplicaDown fires on a lost replica	original: never; fixed: 10–11 s	FAIL → PASS	no rule	–
E4c	ApiHighErrorRate fires	16–17 s	PASS	no rule	–
E4d	only ApiHighLatencyP95 fires	18–19 s, nothing else	PASS	no rule	–
E5	401 / 404, data unchanged, audited	as expected, 11 app_logs rows	PASS	–	–
E6	400 / 413 / 429, service stable	as expected, probe 43/43	PASS	X-Real-IP spoofable	FAIL (finding)
E7	200 rows, 0 duplicates, balances ok	yes, 0 client errors	PASS	yes, after 7 retries	PASS
E8	KZT 201, foreign 422, nothing wrong written	as expected	PASS	–	–
Final	all accounts consistent	35/35, 0 duplicates	PASS	(shared DB)	PASS

9. Discussion, Limitations and Lessons Learned

9.1 Discussion

The measurements support three claims. First, redundancy plus a retrying gateway turns an instance failure from a 23 s outage into no visible failure (E1, E7). Second, monitoring with explicit thresholds detects dependency failures, pod hangs and contention within 10–23 s and stays silent when nothing is wrong (E4). Third, correctness held under every injected fault: no lost or duplicated record in any of the integrity checks. The database outage shows that resilience mechanisms have costs: bounded latency and explicit errors were bought with lower success during and right after the outage (E2). Which is better depends on the client: for a mobile client with its own retry queue, a quick 503 is preferable to a 12 s hang; for a single request that can wait, the baseline's behaviour delivered more answers.

9.2 Limitations

- **The /health endpoint does not check the database.** Liveness is correct, but readiness stays green while PostgreSQL is down (E2) and, likely, while a fresh pod's pool is starved (E1 baseline). A readiness check with a DB ping was rejected in A3 because it would take every replica out at once during a DB outage; a better option is a separate startup gate.
- **No statement timeout and no pool-acquire timeout.** Requests wait on a blocked database until the gateway gives up at 5 s or the server's 15 s write timeout.
- **500 instead of 503 during a DB outage.** The membership check fails closed with 500; the gateway only converts timeouts and 502/503/504 into its JSON 503, so 1–19 responses per outage window were 500s, and gateway retries cover GET only (POST is never replayed, by design).
- **Slower recovery after a DB outage** than the baseline (6–17 s vs 2–13 s): the price of the 5 s cap and `max_fails=1`.
- **Rate limits and login lockout live in each replica's memory**: 10 attempts instead of 5 with two replicas; a restart clears them. All host traffic shares one SNAT IP, so per-IP limits are shared by every test client — this is also why the baseline was throttled in E3.
- **PostgreSQL was OOM-killed once** (256 Mi limit with up to five connection pools). It recovered automatically, but the limit is too tight; there is no database replica.
- **Measurement scope:** one node, latencies include host port forwarding, 2–3 repetitions per scenario (behaviour, not tight distributions); the root cause of the baseline's 23 s outage is a hypothesis; E2 reps 3–4 were invalid and re-run; E8 covered only the empty-archive path; no metrics-server (CPU from kubelet/Prometheus).

9.3 Lessons learned

- An alert must be tested by injecting the failure it is meant to catch: `up == 0` looked right in review and could never fire with DNS-based discovery.
- Verify that a fault was actually injected before reading the result: two "freeze" runs measured nothing because of shell quoting; the alert silence was the clue.
- A load balancer needs more than one upstream to retry on: behind a single ClusterIP the gateway's retry settings were inert until per-pod upstreams were used.
- Proxy headers are part of the security boundary: without the gateway, the client chose its own IP for rate limiting and logs.
- Idempotency keys are cheap and decisive: they made every client retry safe in E2 and E7.

10. Conclusion

The endterm system extends the midterm Registration Service with a gateway, replica-aware routing, an opt-in metrics endpoint, three tested alert rules, a dashboard, request correlation and network segmentation, and it was evaluated in eight experiments against a retained baseline. It meets its acceptance criteria for instance failure, load, monitoring, tenant isolation, input abuse, write consistency and dependency degradation. The comparison is not

uniformly in its favour: during a database outage it trades availability and recovery speed for bounded latency and explicit errors. The main open items are a database-aware startup/readiness gate, a statement timeout, shared rate-limit state, a larger database memory limit and a database replica.

References

[1] Kubernetes Documentation, "Configure Liveness, Readiness and Startup Probes." https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/

[2] Kubernetes Documentation, "Deployments" (rolling update strategy). https://kubernetes.io/docs/concepts/workloads/controllers/deployment/

[3] Kubernetes Documentation, "Pod Lifecycle" (termination, preStop hooks). https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/

[4] Kubernetes Documentation, "Network Policies." https://kubernetes.io/docs/concepts/services-networking/network-policies/

[5] Prometheus Documentation, "Alerting rules." https://prometheus.io/docs/prometheus/latest/configuration/alerting_rules/

[6] Prometheus Documentation, "Histograms and summaries." https://prometheus.io/docs/practices/histograms/

[7] Prometheus Documentation, "Configuration" (dns_sd_config). https://prometheus.io/docs/prometheus/latest/configuration/configuration/

[8] nginx Documentation, "Module ngx_http_upstream_module." https://nginx.org/en/docs/http/ngx_http_upstream_module.html

[9] nginx Documentation, "Module ngx_http_proxy_module." https://nginx.org/en/docs/http/ngx_http_proxy_module.html

[10] M. T. Nygard, Release It! Design and Deploy Production-Ready Software, 2nd ed. Pragmatic Bookshelf, 2018.

[11] B. Beyer, C. Jones, J. Petoff and N. R. Murphy (eds.), Site Reliability Engineering: How Google Runs Production Systems. O'Reilly, 2016 (ch. 6 "Monitoring Distributed Systems", ch. 22 "Addressing Cascading Failures").

[12] OWASP Foundation, "OWASP API Security Top 10 – 2023." https://owasp.org/API-Security/editions/2023/en/0x11-t10/

11. Appendix — Source, Configuration, Scripts, Logs

A. Repository and run instructions

Repository: **https://github.com/shamilstud/howallet-ftr-endterm**. It contains the lab (manifests, scripts, notes, evidence) and the service patch `patch/feat-metrics.diff` (hoWallet commit `60cd5791`, branch `feat/metrics`, base `06aea177`; metrics are off unless `METRICS_ENABLED=true`). The hoWallet application source itself is not modified by the lab.

```
bash scripts/up.sh --baseline          # kind cluster, images, manifests in order, wait for readiness
python scripts/setup_users.py          # synthetic users: register -> verify in SQL -> login (tokens git-ignored)
python scripts/faults.py e1            # E1 replica failure, 3 reps per config
python scripts/faults.py e2            # E2 database outage (stop / freeze), both configs at once
python scripts/exps.py e3 endterm 1    # E3 load step   (also: e3 baseline N)
python scripts/exps.py e4quiet | e4replica N | e4errors N | e4latency N
python scripts/exps.py e5 | e6 | e7 <endterm|baseline> N | e8
python scripts/final_check.py          # final SQL integrity
python scripts/build_summary.py        # evidence/results/summary-data.md from the JSON
bash scripts/down.sh                   # delete the cluster
```

Demonstration plan for the defence (scripted, about 2 minutes each): E1 replica failure with the Grafana dashboard open, E4b hung replica with the alert page, E5/E6 security and misuse requests.

B. Configuration files

Table 21. Manifests.

File	Content
k8s/00-namespace.yaml	namespace university, synthetic Secret university-secrets
k8s/10-postgres.yaml	PostgreSQL 16 StatefulSet, PVC, Service
k8s/20-migrate-job.yaml	migration Job (image ftr-migrate:endterm, Dockerfile.migrate)

File	Content
k8s/30-api.yaml	api Deployment x2, Service api, headless Service api-headless
k8s/40-gateway.yaml	nginx ConfigMap (Listing 1), gateway Deployment x2, NodePort 30080
k8s/50-prometheus.yaml	scrape config (dns_sd, 2 s) and the three alert rules
k8s/60-grafana.yaml	datasource and dashboard howallet-endterm (7 panels)
k8s/70-network-policy.yaml	default-deny ingress, per-tier allow rules, API egress allow-list
k8s/baseline/api-baseline.yaml	midterm baseline: 1 replica, NodePort 30081, own headless Service
k8s/optional/block-egress.yaml	replacement egress policy without external HTTPS
kind-config.yaml	single-node cluster with NodePort mappings 30080/30081/30090/30300

C. Scripts

Table 22. Scripts.

File	Purpose
scripts/up.sh, down.sh, switch-api-image.sh, get-token.sh, rec.sh	cluster lifecycle, token, recording `$ command` + output to evidence/*.txt
scripts/ftlab.py	Probe, EndpointWatcher, AlertWatcher, summarize, known_state
scripts/faults.py	E1, E2
scripts/exps.py	E3–E8
scripts/e1_diag.sh, e1_diag2.py, e1_diag3.py	E1 baseline diagnostics
scripts/setup_users.py, final_check.py, build_summary.py	users, final integrity, summary tables
scripts/render_evidence.py, shoot.mjs, architecture.html	terminal renders, browser screenshots, architecture diagram
report/build_report.py	this report (DOCX + PDF) from the evidence

D. Logs and additional results

Text logs: `evidence/01-cluster-up.txt`, `02-get-all.txt`, `03-targets.txt`, `04-correlation.txt`, `05-alert-rules.txt`, `e1-replica-failure.txt`, `e1-baseline-diagnostic.txt`, `e2-database-outage.txt`, `e3-load-increase.txt`, `e4-alerts.txt`, `e5-access-control.txt`, `e6-input-limits.txt`, `e7-recovery-consistency.txt`, `e8-graceful-degradation.txt`, `99-final-integrity.txt`. Raw data: `evidence/results/*.json` (31 files incl. the two `*-INVALID-not-injected.json`), `summary.md`, `summary-data.md`. Note: the terminal logs also print a `stats_interval` that includes the 5 s before the injection; the tables in this report use `stats_after_injection` (E1) and the windows of Table 11.

Screenshots in `evidence/screenshots/` not reproduced (or reproduced only as a crop) in the body, besides the `terminal-*--main.png` renders of each command header:

`prom-alerts-e4-error-rate-rep1-ApiHighErrorRate-firing.png`, `prom-alerts-e4-error-rate-rep1-ApiHighLatencyP95-firing.png`, `prom-alerts-e4-error-rate-rep2-ApiHighErrorRate-firing.png`, `prom-alerts-e4-error-rate-rep2-ApiHighLatencyP95-firing.png`, `prom-alerts-e4-latency-rep1-ApiHighLatencyP95-firing.png`, `prom-alerts-e4-latency-rep2-ApiHighLatencyP95-firing.png`, `prom-alerts-e4-replica-down-rep2-ApiReplicaDown-firing.png`, `prom-alerts-e4-replica-down-rep3-ApiReplicaDown-firing.png`, `prometheus-targets.png`, `terminal-01-cluster-up.png`, `terminal-02-get-all.png`, `terminal-03-targets.png`, `terminal-04-correlation.png`, `terminal-05-alert-rules.png`, `terminal-e1-baseline-diagnostic.png`, `terminal-e1-replica-failure--e1-baseline-rep1-graceful.png`, `terminal-e1-replica-failure--e1-baseline-rep2-graceful.png`, `terminal-e1-replica-failure--e1-baseline-rep3-crash.png`, `terminal-e1-replica-failure--e1-endterm-rep1-graceful.png`, `terminal-e1-replica-failure--e1-endterm-rep2-graceful.png`, `terminal-e1-replica-failure--e1-endterm-rep3-crash.png`, `terminal-e2-database-outage--e2-rep1-stop-part1.png`, `terminal-e2-database-outage--e2-rep1-stop-part2.png`, `terminal-e2-database-outage--e2-rep2-stop-part1.png`, `terminal-e2-database-outage--e2-rep2-stop-part2.png`, `terminal-e2-database-outage--e2-rep3-freeze.png`, `terminal-e2-database-outage--e2-rep4-freeze.png`, `terminal-e2-database-outage--e2-rep5-freeze-part1.png`, `terminal-e2-database-outage--e2-rep5-freeze-part2.png`, `terminal-e2-database-outage--e2-rep6-freeze-part1.png`, `terminal-e2-database-outage--e2-rep6-freeze-part2.png`, `terminal-e3-load-increase--e3-baseline-rep1.png`, `terminal-e3-load-increase--e3-baseline-rep2.png`, `terminal-e3-load-increase--e3-endterm-rep1.png`, `terminal-e3-load-increase--e3-endterm-rep2.png`, `terminal-e4-alerts--e4-error-rate-rep1.png`, `terminal-e4-alerts--e4-error-rate-rep2.png`, `terminal-e4-alerts--e4-latency-rep1.png`, `terminal-e4-alerts--e4-latency-rep2.png`, `terminal-e4-alerts--e4-quiet-window.png`, `terminal-e4-alerts--e4-replica-down-rep1.png`, `terminal-e4-alerts--e4-replica-down-rep2.png`, `terminal-e4-alerts--e4-replica-down-rep3.png`, `terminal-e5-access-control--E5-access-control.png`, `terminal-e7-recovery-consistency--e7-baseline-rep1.png`, `terminal-e7-recovery-consistency--e7-baseline-rep2.png`, `terminal-e7-recovery-consistency--e7-endterm-rep1.png`, `terminal-e7-recovery-consistency--e7-endterm-rep2.png`, `terminal-e8-graceful-degradation--E8-graceful-degradation-exchange-rate-provider-unreachable.png`